

```
'missing!'
```

This very nature differentiates a `defaultdict` from a regular Dictionary. When a `dict` is unable to find a key it will raise a `KeyError` instead of returning a default value. A `defaultdict` too will raise a `KeyError` in case we pass `None` as a `default_factory`.

Take due care to ensure that you are passing a function reference to the constructor and not a function call.

```
>>> attendance = defaultdict(constant_fac-
tory(), {'ram': 'p', 'shayam': 'a', 'rahul':
'p', 'ria': 'p', 'neha': 'a'});
```

The above piece of code will raise an error saying, `TypeError: first argument must be callable or None`, since we passed a method call and not a method reference to the constructor.

We can also use a `lambda` to pass function references in the constructor. `lambdas` are anonymous functions in python that are defined without a name. If we wish to supply a lambda function in the above example, it would look something like this:

```
>>> attendance = defaultdict(lambda:
'missing!' , {'ram': 'p',
'shayam': 'a', 'rahul': 'p', 'ria': 'p', 'neha': 'a'});
```

As you can see there was no need of defining a separate function which we pass as a `default_factory`, rather an inline `lambda` was more than sufficient.

A `defaultdict` can be polymorphically used as a lot of data structures based on how we build the default factory. We already saw how it can be used in place of a `Counter` under the examples of Counters. In the next example, we will see how it can be used as a dictionary of sets.

```
>>> s=[('blue', 2), ('red', 3), ('blue', 4), ('red',
1), ('blue', 4)]
>>> d = defaultdict(set)
>>> for key, value in s:
...     d[key].add(value)
...
>>> sorted(d.items())
[('blue', {2, 4}), ('red', {1, 3})]
```

Hence, a `defaultdict` can be interchangeably used with a Dictionary when each element requires a default value.